

Tên: Cao Minh Thuận

MSSV: 24521734

BÀI THỰC HÀNH SỐ 1

1. Lý thuyết

Giảng viên hướng dẫn sinh viên sử dụng phần mềm MARS dựa theo tài liệu: **MARS – chương trình mô phỏng hợp ngữ (assembly) MIPS**

2. Thực hành

2.1 Sinh viên tìm hiểu tài liệu “**Một số lệnh assembly MIPS cơ bản**” và mô phỏng việc thực thi các lệnh và cho biết chức năng của các lệnh cơ bản sau:

add, addi, addu, addiu, sub, subu, and, andi, or, nor, lw, sw, slt, slti, sltu, sltiu

***Lệnh add:** add \$t0, \$t1, \$t2 (R-format)

- Chức năng: cộng giá trị của 2 thanh ghi t1 và t2, sau đó lưu kết quả vào thanh ghi t0

***Lệnh addi:** addi \$t0, \$t1, 1 (I-format)

- Chức năng: cộng giá trị của thanh ghi t1 với số tức thời (phạm vi số tức thời trong lệnh này là 16 bit) , sau đó lưu kết quả vào thanh ghi t0

***Lệnh addu và addiu:** addu \$t0, \$t1, \$t2 (R-format) và addiu \$t0, \$t1, 1 (I-format)

- Lệnh addu có chức năng giống lệnh add còn lệnh addiu có chức năng giống lệnh addi

Tuy nhiên, addiu và addu không xét kết quả có bị overflow hay không, trong khi đó addi và add sẽ báo khi overflow xuất hiện

***Lệnh sub:** sub \$t0, \$t1, \$t2 (R-format)

- Chức năng: lấy giá trị của thanh ghi t1 trừ cho giá trị của thanh ghi t2, sau đó lưu kết quả vào thanh ghi t0

***Lệnh subu:** subu \$t0, \$t1, \$t2 (R-format)

Lệnh subu có chức năng giống lệnh sub nhưng lệnh subu không xét đến kết quả có bị overflow hay không, trong khi đó lệnh sub có xét đến kết quả có bị overflow hay không, nếu bị overflow, sẽ có thông báo

***Lệnh and:** and \$t0, \$t1, \$t2 (R-format)

Chức năng: thực hiện and từng bit giá trị của 2 thanh ghi t1 và t2 lại với nhau, sau đó lưu kết quả vào thanh ghi t0

***Lệnh andi:** andi \$t0, \$t1, 1 (I-format)

Chức năng: thực hiện and từng bit giá trị của thanh ghi t1 và một số tức thời (số tức thời đang là số 16 bits), sau đó lưu giá trị vào thanh ghi t0

***Lệnh or:** or \$t0, \$t1, \$t2 (R-format)

Chức năng: thực hiện or từng bit giá trị 2 thanh ghi t1 và t2 lại với nhau, sau đó lưu kết quả vào thanh ghi t0

***Lệnh nor:** nor \$t0, \$t1, \$t2 (R-format)

Chức năng: thực hiện nor từng bit giá trị 2 thanh ghi t1 và t2 lại với nhau, sau đó lưu kết quả vào thanh ghi t0

***Lệnh lw:** lw \$t1, 4(\$t0) (I-format)

Chức năng: Lấy giá trị trong thanh ghi t0 cộng với số tức thời ta được địa chỉ của từ nhớ cần lấy dữ liệu, dữ liệu của từ nhớ này sẽ được lấy để lưu vào thanh ghi t1

***Lệnh sw:** sw \$t1, 8(\$t0) (I-format)

Chức năng: Lưu giá trị thanh ghi t1 vào từ nhớ có địa chỉ được tính bằng giá trị thanh ghi t0 cộng với offset (offset được mở rộng có dấu thành số 32 bits trước khi cộng)

***Lệnh slt và lệnh sltu:** slt \$t0, \$t1, \$t2 (R-format) và sltu \$t0, \$t1, \$t2 (R-format)

Chức năng: cả hai lệnh đều thực hiện kiểm tra thanh ghi t1 có nhỏ hơn thanh ghi t2 hay không, nếu nhỏ hơn thì thanh ghi t0 được nhận giá trị là 1, ngược lại thanh ghi t0 nhận giá trị là 0

Lưu ý: lệnh slt được dùng để kiểm tra trên số có dấu, còn lệnh sltu được dùng để kiểm tra trên số không có dấu

***Lệnh slti và lệnh sltiu:** slti \$t0, \$t1, 0x73 (I-format) và sltiu \$t0, \$t1, 0x83(I-format)

Chức năng: cả 2 lệnh trên đều thực hiện so sánh thanh ghi t1 với một số tức thời, nếu giá trị thanh ghi t1 nhỏ hơn số tức thời thì thanh ghi t0 nhận giá trị là 1, ngược lại thanh ghi t0 nhận giá trị là 0

Lưu ý: lệnh slti là so sánh 2 giá trị theo kiểu có dấu dạng bù 2, trong khi đó sltiu so sánh theo kiểu số không dấu

2.2. Mô phỏng các chương trình bên dưới và có biết ý nghĩa của chương trình:

Ví dụ 1:

```
.data
var1: .word 23

.text
__start:
    lw    $t0, var1
    li    $t1, 5
    sw    $t1, var1
```

Ý nghĩa của chương trình:

-“var1: .word 23” : Khai báo biến var1 có kiểu dữ liệu là word trong bộ nhớ, giá trị ban đầu là 23

-“lw \$t0, var1”: Đưa giá trị của biến var1 vào thanh ghi t0

-“li \$t1, 5”: Đặt giá trị của thanh ghi t1 thành 5

-“sw \$t1, var1”: ghi giá trị trong thanh ghi t1 vào ô nhớ của biến var1

Ví dụ 2:

```
array1:      .data
             .space 12
             .text
__start:     la      $t0, array1
             li      $t1, 5
             sw $t1, ($t0)
             li $t1, 13
             sw $t1, 4($t0)
             li $t1, -7
             sw $t1, 8($t0)
```

Ý nghĩa của chương trình:

-“array1: .space 12”: Khai báo một mảng array1 trong bộ nhớ có 12 bytes liên tục

-“la \$t0, array1”: Đưa địa chỉ của mảng array1 vào thanh ghi t0

-“li \$t1, 5”: Đặt giá trị của thanh ghi t1 thành 5

-“sw \$t1, (\$t0)”: Ghi giá trị của thanh ghi t1 vào bộ nhớ có địa chỉ được lưu ở thanh ghi t0

-“li \$t1, 13”: Cập nhật giá trị của thanh ghi t1 thành 13

-“sw \$t1, 4(\$t0)” : Ghi giá trị của thanh ghi t1 vừa mới cập nhật vào địa chỉ được lưu ở thanh ghi t0 + 4

-“li \$t1, -7”: Cập nhật giá trị của thanh ghi t1 thành -7

-“sw \$t1, 8(\$t0)” : Ghi giá trị của thanh ghi t1 vừa mới cập nhật vào địa chỉ được lưu ở thanh ghi t0 + 8

Ví dụ 3:

```
li      $v0, 5
syscall
```

Ý nghĩa của chương trình:

-“li \$v0, 5”: Đặt giá trị của thanh ghi v0 thành 5

-“syscall”: Nhập input từ bàn phím

Ví dụ 4:

```
string1:      .data
              .ascii "Print this.\n"

main:         .text
              li      $v0, 4
              la      $a0, string1
              syscall
```

Ý nghĩa của chương trình:

- “string1: .asciiz “Print this.\n””: Khai báo biến string1 trong bộ nhớ có chứa chuỗi “Print this .\n”
- “li \$v0, 4”: Đặt giá trị của thanh ghi v0 thành 4
- “la \$a0, string1”: Đưa địa chỉ của biến string1 vào thanh ghi a0
- “syscall” : In chuỗi “Print this.”ra màn hình

2.3 Nhập vào một chuỗi, xuất ra cửa sổ I/O của MARS theo từng yêu cầu sau:

a) Khai báo và xuất ra cửa sổ I/O 2 chuỗi có giá trị như sau:

- Chuỗi 1: Chao ban! Ban la sinh vien nam thu may?
- Chuỗi 2: Hihi, minh la sinh vien nam thu 1 ^-^

```
.data
string1: .asciiz "Chao ban! Ban la sinh vien nam thu may?\n"
string2: .asciiz "Hihi, minh la sinh vien nam thu 1 ^-^\n"
```

```
.text
main:   li      $v0, 4

        la      $a0, string1
        syscall

        la      $a0, string2
        syscall
```

b) Biểu diễn của 2 chuỗi trên dưới bộ nhớ là gì?

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x6f616843	0x6e616220	0x61422021	0x616c206e	0x6e697320	0x6e620665	0x6e682076	0x69656976
0x10010020	0x6d207568	0x69687961	0x68694800	0x6d202c69	0x20686e69	0x7320616c	0x20686e69	0x6e656976
0x10010040	0x6d616e20	0x75687420	0x5e203120	0x000a5e2d	0x00000000	0x00000000	0x00000000	0x00000000

String1: 43 68 61 6F 20 62 61 6E 21 20 42 61 6E 20 6C 61 20 73 69 6E 68 20 76 69 65 6E 20 6E 61 6D 20 74 68 75 20 6D 61 79 3F 0A 00

String2: 48 69 68 69 2C 20 6D 69 6E 68 20 6C 61 20 73 69 6E 68 20 76 69 65 6E 20 6E 61 6D 20 74 68 75 20 31 20 5E 2D 5E 0A 00

c) Xuất ra lại đúng chuỗi đã nhập

```
.data
string: .space 100

.text

main:

    li $v0,8
    la $a0, string
    li $a1,100
    syscall

    li $v0,4
    la $a0, string
    syscall
```

Ví dụ:

Nhap: Truong Dai hoc Cong nghe Thong tin

Xuat: Truong Dai hoc Cong nghe Thong tin

d) Nhập vào 2 số nguyên sau đó xuất tổng của 2 số nguyên này

```
.data
so1: .asciiz "nhap so1: "
so2: .asciiz "nhap so2: "
tong: .asciiz "tong = "

.text

main:

    li $v0,4
    la $a0,so1
    syscall

    li $v0,5
```

```
syscall  
move $t0,$v0
```

```
li $v0,4  
la $a0,so2  
syscall
```

```
li $v0,5  
syscall  
move $t1,$v0
```

```
li $v0,4  
la $a0,tong  
syscall
```

```
add $t3,$t0,$t1
```

```
li $v0,1  
move $a0,$t3  
syscall
```